

PRIMER AVANCE

Primer avance



PRESENTADO POR: MARCKUS ALBERTO RAMIREZ
REYNA | A01384881
ARMANDO JAVIER FLORES SALAZAR |
A00844090

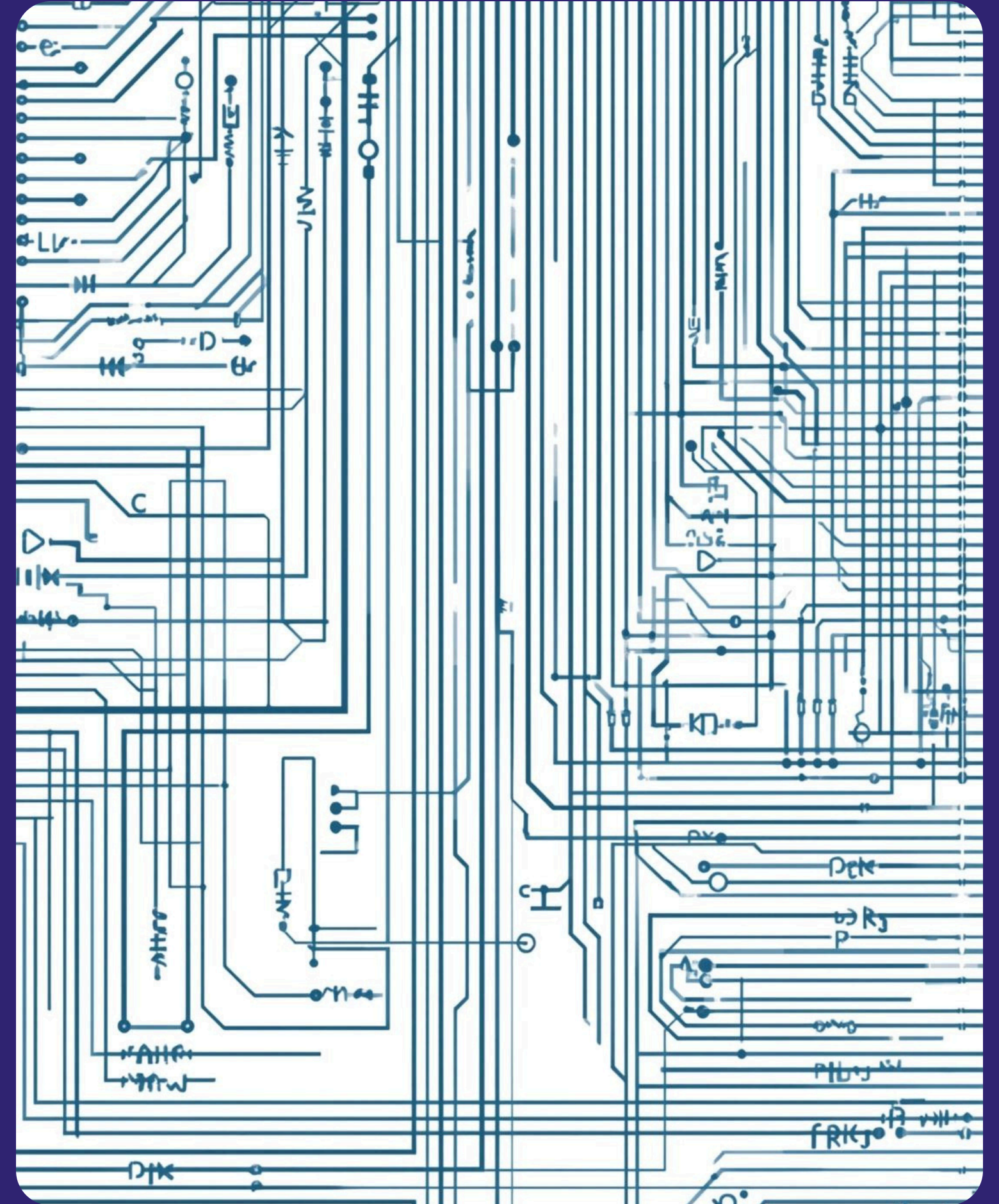
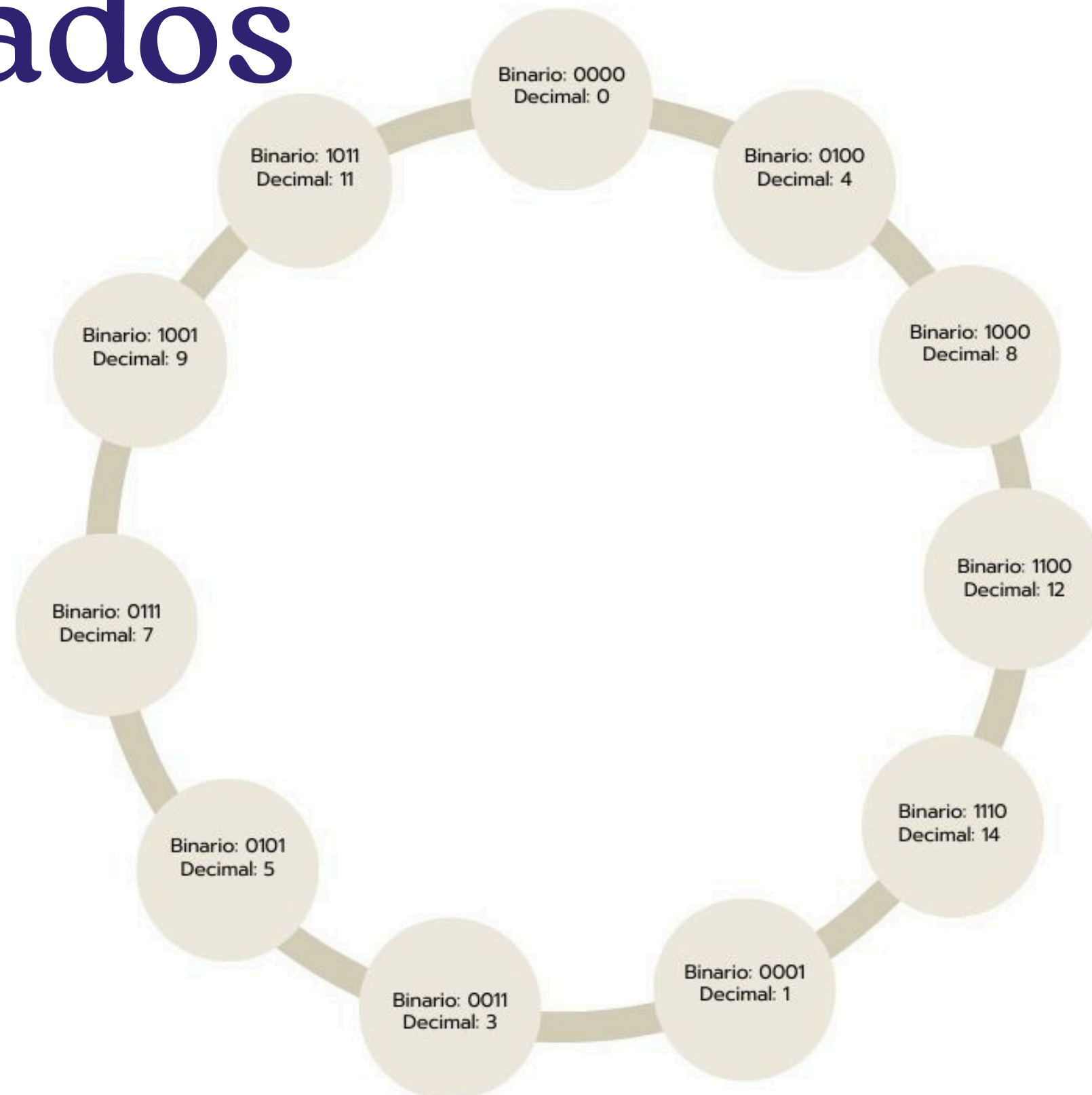


Diagrama de estados



Representación gráfica de estados

La secuencia de estados se **visualiza claramente** en un diagrama, donde cada estado está representado por un círculo, facilitando la comprensión del circuito secuencial.

Estados codificados en 4 bits

Los estados en el diagrama están codificados en 4 bits (Q3 Q2 Q1 Q0), lo que permite una representación compacta y eficaz de la lógica del circuito.

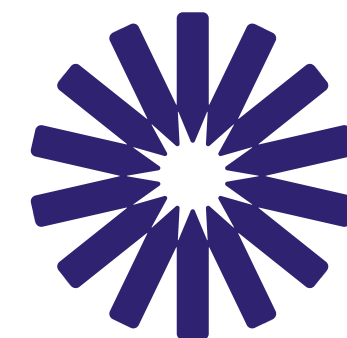


Tabla de transición

ESTADO ACTUAL				ESTADO SIGUIENTE			
Q3	Q2	Q1	Q0	Q3 ⁺	Q2 ⁺	Q1 ⁺	Q0 ⁺
0	0	0	0	0	1	0	0
0	1	0	0	1	0	0	0
1	0	0	0	1	1	0	0
1	1	0	0	1	1	1	0
1	1	1	0	0	0	0	1
0	0	0	1	0	0	1	1
0	0	1	1	0	1	0	1
0	1	0	1	0	1	1	1
0	1	1	1	1	0	0	1
1	0	0	1	1	0	1	1
1	0	1	1	0	0	0	0

ESTADOS NO USADOS			
Q3	Q2	Q1	Q0
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1
1	1	1	1

Estado presente y siguiente en secuencia

Esta tabla ilustra la **relación** entre los estados presentes y siguientes, mostrando la secuencia definida en el circuito.

Tabla de excitación JK

ESTADO ACTUAL				ESTADO SIGUIENTE				FLIP-FLOP 3		FLIP-FLOP 2		FLIP-FLOP 1		FLIP-FLOP 0	
Q3	Q2	Q1	Q0	Q3 ⁺	Q2 ⁺	Q1 ⁺	Q0 ⁺	J3	K3	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	0	0 X		1 X		0 X		0 X	
0	1	0	0	1	0	0	0	1 X		X	1	0 X		0 X	
1	0	0	0	1	1	0	0 X		0	1 X		0 X		0 X	
1	1	0	0	1	1	1	0 X		0 X		0	1 X		0 X	
1	1	1	0	0	0	0	1 X		1 X		1 X		1	1 X	
0	0	0	1	0	0	1	1	0 X		0 X		1 X	X		0
0	0	1	1	0	1	0	1	0 X		1 X	X		1 X		0
0	1	0	1	0	1	1	1	0 X	X		0	1 X	X		0
0	1	1	1	1	0	0	1	1 X	X		1 X		1 X		0
1	0	0	1	1	0	1	1 X		0	0 X		1 X	X		0
1	0	1	1	0	0	0	0 X		1	0 X	X		1 X		1

Estados presentes y entradas de flip-flops

La tabla de excitación JK muestra **visualmente** cómo se relacionan los estados y entradas J y K de cada flip-flop.

Simplificación con Mapas

MAPA J0						MAPA K0					
Q1-Q0						Q1-Q0					
		00	01	11	10			0	1	11	10
Q3-Q2	00	0	X	X	X	Q3-Q2	0	0	0	0	X
	01	0	X	X	X		1	0	0	0	X
	11	0	X	X	1		11	0	X	X	X
	10	0	X	X	X		10	0	0	1	X

MAPA J1						MAPA K1					
Q1-Q0						Q1-Q0					
		00	01	11	10			00	01	11	10
Q3-Q2	00	0	1	0	X	Q3-Q2	00	0	0	1	X
	01	0	1	0	X		01	0	0	1	X
	11	1	X	X	X		11	0	X	X	1
	10	0	1	0	X		10	0	0	1	X

MAPA J2						MAPA K2					
Q1-Q0						Q1-Q0					
		00	01	11	10			00	01	11	10
Q3-Q2	00	1	0	1	X	Q3-Q2	00	0	0	0	X
	01	0	0	0	X		01	1	0	1	X
	11	0	X	X	X		11	0	X	X	1
	10	1	0	0	X		10	0	0	0	X

MAPA J3						MAPA K3					
Q1-Q0						Q1-Q0					
		00	01	11	10		Q3Q2\ Q1Q0	00	01	11	10
Q3-Q2	00	0	0	0	X	Q3-Q2	00	0	0	0	X
	01	1	0	1	X		01	0	0	0	X
	11	0	X	X	0		11	0	X	X	1
	10	0	0	0	X		10	0	0	1	X

Expresiones Booleanas

Entrada	Expresión Booleana Simplificada
J3	$Q2 \cdot Q1 + Q2 \cdot Q0'$
K3	$Q1 \cdot Q2 + Q1 \cdot Q0 = Q1(Q2 + Q0)$
J2	$Q1' \cdot Q0' + Q3' \cdot Q1 \cdot Q0$
K2	$Q3' \cdot Q2 \cdot Q0' + Q2 \cdot Q1 = Q2(Q3' \cdot Q0' + Q1)$
J1	$Q1' \cdot Q0 + Q3 \cdot Q2$
K1	$Q1 \cdot Q0 + Q1 \cdot Q2 = Q1(Q0 + Q2)$
J0	$Q2 \cdot Q1$
K0	$Q3 \cdot Q1$

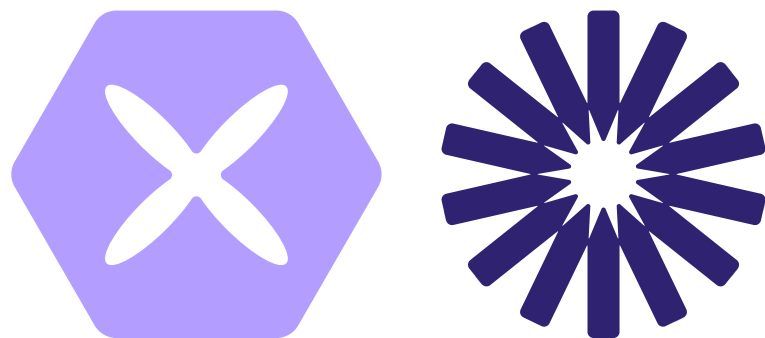


Diseño del circuito

El circuito secuencial consta de cuatro flip-flops JK (FF3, FF2, FF1, FF0) sincronizados por un reloj común (CLK), y un circuito combinatorio que calcula las entradas J y K con compuertas AND, OR y NOT.

Compuertas del circuito combinatorio:

- $J_3 = Q_2 \cdot Q_1 + Q_2 \cdot Q_0' \rightarrow \text{NOT}(Q_0), \text{AND2}(Q_2, Q_1), \text{AND2}(Q_2, Q_0'), \text{OR2}$
- $K_3 = Q_1 \cdot (Q_2 + Q_0) \rightarrow \text{OR2}(Q_2, Q_0), \text{AND2}(Q_1, \text{OR_out})$
- $J_2 = Q_1' \cdot Q_0' + Q_3' \cdot Q_1 \cdot Q_0 \rightarrow \text{NOT}(Q_1), \text{NOT}(Q_0), \text{NOT}(Q_3), \text{AND2}(Q_1', Q_0'), \text{AND3}(Q_3', Q_1, Q_0), \text{OR2}$
- $K_2 = Q_2 \cdot (Q_3' \cdot Q_0' + Q_1) \rightarrow \text{NOT}(Q_3), \text{NOT}(Q_0), \text{AND2}(Q_3', Q_0'), \text{OR2}(\text{AND}, Q_1), \text{AND2}(Q_2, \text{OR_out})$
- $J_1 = Q_1' \cdot Q_0 + Q_3 \cdot Q_2 \rightarrow \text{NOT}(Q_1), \text{AND2}(Q_1', Q_0), \text{AND2}(Q_3, Q_2), \text{OR2}$
- $K_1 = Q_1 \cdot (Q_0 + Q_2) \rightarrow \text{OR2}(Q_0, Q_2), \text{AND2}(Q_1, \text{OR_out})$
- $J_0 = Q_2 \cdot Q_1 \rightarrow \text{AND2}(Q_2, Q_1)$
- $K_0 = Q_3 \cdot Q_1 \rightarrow \text{AND2}(Q_3, Q_1)$



4 Flip-Flops JK sincronizados

Los **Flip-Flops JK** están sincronizados mediante una señal CLK, permitiendo el almacenamiento y la transición eficiente entre diferentes estados en el circuito secuencial diseñado.

Circuito combinatorio con puertas

Se implementan puertas **AND**, **OR** y **NOT** para crear un circuito combinatorio que procesa las entradas J y K, ajustando la salida del sistema según la lógica definida.

Simulación en Python

```
def ff_jk(q, j, k):  
    if j == 0 and k == 0:  
        return q  
    if j == 0 and k == 1:  
        return 0  
    if j == 1 and k == 0:  
        return 1  
    if j == 1 and k == 1:  
        return 1 - q
```

```
def calcular_jk(q3, q2, q1, q0):  
    j3 = int((q2 and q1) or (q2 and not q0))  
    k3 = int((q1 and q2) or (q1 and q0))  
    j2 = int((not q1 and not q0) or (not q3 and q1 and q0))  
    k2 = int((not q3 and q2 and not q0) or (q2 and q1))  
    j1 = int((not q1 and q0) or (q3 and q2))  
    k1 = int((q1 and q0) or (q1 and q2))  
    j0 = int(q2 and q1)  
    k0 = int(q3 and q1)
```

```
    return j3, k3, j2, k2, j1, k1, j0, k0
```

```
def simular(estado_inicial=0, ciclos=12):  
    q3 = (estado_inicial >> 3) & 1  
    q2 = (estado_inicial >> 2) & 1  
    q1 = (estado_inicial >> 1) & 1  
    q0 = estado_inicial & 1
```

```
    for c in range(ciclos):  
        decimal = q3*8 + q2*4 + q1*2 + q0
```

```
        print("Ciclo", c+1, "-", q3, q2, q1, q0, " dec:", decimal)
```

```
        j3, k3, j2, k2, j1, k1, j0, k0 = calcular_jk(q3, q2, q1, q0)
```

```
        q3 = ff_jk(q3, j3, k3)  
        q2 = ff_jk(q2, j2, k2)  
        q1 = ff_jk(q1, j1, k1)  
        q0 = ff_jk(q0, j0, k0)
```

```
    print("Termino en:", q3, q2, q1, q0 )
```

```
simular(0, 12)
```

```
fif_adcalel);  
vc('urm= noblildrare'""fkbe]nops")  
rr;3oit bripsiptios );  
fit frm-JIe_mig (u0n lot3)  
ldir{ {  
    ffit :fnmill/diap;  
    va cwit.louws.rumre(octekcopa) ]  
    to "tattion" im _mO  
    ajflgJbllp3(arneec(as|eal)!'  
    _pit()  
    va "pit-(aniptige [ ;;  
    f eoblp jabs(irckilola );  
    faleibakern(mbatluceEutt(A.ops)  
    liit-er-jergditllced0);  
    sintle whad 00nJ/Kbinc(ard))  
    )  
};  
jã)
```

Implementación de función y cálculo de entradas

Esta sección muestra la implementación de la función **ff_jk** en Python, calculando entradas J y K para simular el comportamiento del circuito.